

Formalising Mathematics:

Proving basic notions of limits and boundedness

Joan Arenillas i Cases¹

¹MSc in Applied Mathematics, Imperial College London.

CID: 06067859, ja1225@ic.ac.uk

Abstract. In this report I state elemental definitions of limits of functions and prove that if a function f has a limit at some point x_0 of its domain, then f is necessarily bounded on some neighbourhood of x_0 . In the way to prove this first-year result, we define concepts on limits, working through some examples. In the second part of the report we formalise our main result, as well as other auxiliary—but necessary—definitions. At the end, we discuss the challenges faced, reflecting on the assumptions.

1 Introduction

The goal of this report is to formalise in Lean a basic result in real analysis. It states the following:

Lemma: Boundedness around a point

Let $f : (a, b) \rightarrow \mathbb{R}$ for some $a, b \in \mathbb{R}$. If there exists the limit of f at $x_0 \in (a, b)$, then f is bounded on some neighbourhood of x_0 .

We will see in [Section 2](#) that this result can be proved in a few lines with a small number of definitions on limits and boundedness. However, we will discover in [Section 3](#) that the Lean implementation is far more intricate. It will require handling the proof by cases and considering intervals and bounds very carefully. We will ponder over the challenges we faced in [Section 4](#), and we will propose some ways to move forward in [Section 5](#).

There are countless sources explaining the basic properties of real analysis in one variable. Unless otherwise stated, in this report I have followed [\[3\]](#).

2 Informal Proof

To reach the proof of our lemma, we must first define some concepts. Let $f : (a, b) \rightarrow \mathbb{R}$ be a function and consider $x_0 \in (a, b)$. We say that ℓ is the *limit of the function f at x_0* and we write $\lim_{x \rightarrow x_0} f(x) = \ell$ if for all $\varepsilon > 0$ there exists $\delta > 0$ such that $|f(x) - \ell| < \varepsilon$ if $0 < |x - x_0| < \delta$. Observe that here we are always supposing that the limit ℓ is finite and that f does not take infinite values in (a, b) .

Although we will not need this result to prove the desired Lemma we will also introduce the notion of continuity to practise with Lean definitions. Specifically, we will say that the function f is *continuous at $x_0 \in (a, b)$* if the limit $\lim_{x \rightarrow x_0} f(x)$ exists and is equal to $f(x_0)$. Naturally, we will say that f is *continuous on (a, b)* if f is continuous at x_0 for every $x_0 \in (a, b)$. Lastly, we say that $f : \mathbb{R} \rightarrow \mathbb{R}$ is *continuous on $\mathbb{R}$* if it is continuous at every $x_0 \in \mathbb{R}$.

As an example, consider $g(x) = x$ defined for all $x \in \mathbb{R}$. It is now easy to check that $g(x_0) = \lim_{x \rightarrow x_0} g(x) = x_0$ for any $x_0 \in \mathbb{R}$. Indeed, fix some $\varepsilon > 0$ and define $\delta := \varepsilon > 0$. Now, for any $x \in \mathbb{R}$ with $0 < |x - x_0| < \delta = \varepsilon$ we have that $|g(x) - x_0| = |x - x_0| < \varepsilon$.

It is also worth noting that it makes sense to talk about “the” limit of a function f because if it exists, this limit is unique (we will not formally nor informally prove this result).

We will now introduce the fundamental concept for the Lemma we aim to prove in Lean. Let I be an interval and consider $f : I \rightarrow \mathbb{R}$. We say that f is *bounded on I* if there exist some constants $m, M \in \mathbb{R}$ such that $m \leq f(x) \leq M$ for all $x \in I$. This condition can be equivalently stated as follows.

Lemma 2.1: Equivalence of boundedness definitions

A function $f : I \rightarrow \mathbb{R}$ is bounded on I if and only if there exists some real constant $K \geq 0$ such that $|f(x)| \leq K$ for every $x \in I$.

Proof. If f is bounded on I then there exist some real constants m and M such that $m \leq f(x) \leq M$ for all $x \in I$, so that $|f(x)| \leq \max\{|m|, |M|\} =: K$. Conversely, if $|f(x)| \leq K, \forall x \in I$ for some $K \geq 0$, we deduce that $-K \leq f(x) \leq K$ for all $x \in I$. $\square$

In a similar way, we say that $f : I \rightarrow \mathbb{R}$ is bounded on $J \subseteq I$ if its restriction $f|_J$ is bounded on J .

Observe that the existence of the limit of a function in one point $x_0 \in I$ does not provide us with global information about the function, but rather gives information on a neighbourhood of x_0 . We now state and prove the main result we are interested in, which gives information on the function f near x_0 .

Lemma 2.2: Boundedness around a point

Let $f : (a, b) \rightarrow \mathbb{R}$ for some $a, b \in \mathbb{R}$. If there exists the limit of f at $x_0 \in (a, b)$, then f is bounded on some neighbourhood of x_0 .

Proof. Let $\ell := \lim_{x \rightarrow x_0} f(x)$. Let us apply the definition of limit with $\varepsilon = 1$. Then, we have that there exists some $\delta > 0$ such that $|f(x) - \ell| < 1$ if $0 < |x - x_0| < \delta$. Then, by the triangle inequality we have that $|f(x)| = |(f(x) - \ell) + \ell| \leq |f(x) - \ell| + |\ell| < 1 + |\ell|$ if $0 < |x - x_0| < \delta$. To conclude, let $K := \max\{1 + |\ell|, |f(x_0)|\} < \infty$ and note that $K \geq 0$. We then deduce that $|f(x)| \leq K$ if $x \in (x_0 - \delta, x_0 + \delta)$, so that f is bounded on this neighbourhood of x_0 . $\square$

3 Formal Proof

We now turn our attention to formalising [Section 2](#) in Lean¹. We are going to follow the structure of the previous section.

3.1 Preliminary definitions and examples

We start by defining the interval endpoints with `variable (a b : ℝ)` and further defining the notion of limit of a function $f : (a, b) \rightarrow \mathbb{R}$ at a point x_0 (with limit ℓ).

```
1 def limit_f_x_0 {a b : ℝ} (l x_0 : ℝ) (f : Set.Ioo a b → ℝ) : Prop :=
2   ∀ ε > 0, ∃ δ > 0, ∀ x : Set.Ioo a b, |x - x_0| < δ ∧ |x - x_0| > 0 → |f x - l| < ε
```

Listing 1: Definition of limit of f at x_0 .

Observe that the open interval is defined through `Set.Ioo a b`, where we do not need to specify the condition $a < b$ because Lean will consider the empty set if $a > b$. We continue by giving the definition of continuity of f at x_0 , continuity on the whole interval (a, b) and continuity on the real line.

¹I acknowledge that I have used AI tools to check the spelling and grammatical structure of this report. Also, GitHub Copilot has been used to suggest possible tactics and lemmas in the proofs.

```

1  def continuous_x0 {a b : ℝ} (f : Set.Ioo a b → ℝ) (x0 : ℝ) (hx0 : x0 ∈ Set
2  .Ioo a b) : Prop :=
    limit_f_x_0 (λ := f ⟨x0, hx0⟩) x0 f

```

Listing 2: Definition of continuity at x_0 .

```

1  def continuous_on_interval (f : Set.Ioo a b → ℝ) : Prop :=
2  ∀ x0 (hx0 : x0 ∈ Set.Ioo a b), continuous_x0 f x0 hx0

```

Listing 3: Definition of continuity on (a, b) .

```

1  def continuous_on_ℝ (f : ℝ → ℝ) : Prop :=
2  ∀ a b x0 (hx0 : x0 ∈ Set.Ioo a b),
3  continuous_x0 (fun x : Set.Ioo a b => f x) x0 hx0

```

Listing 4: Definition of continuity on all $\mathbb{R}$.

The definitions in Listings 2 to 4 are very standard and closely follow the definitions we introduced in Section 2. It is worth noting, however, the use of $\{a\ b : \mathbb{R}\}$ at the beginning of the definitions of limit and continuity at x_0 . By doing this, we will not need to specify a and b as arguments when using these definitions later in the code.

It is also worth noting that we defined continuity over $\mathbb{R}$ (Listing 4) by using the notion of continuity at a point x_0 we had previously introduced. This forces us to define continuity on $\mathbb{R}$ by saying that a continuous function on $\mathbb{R}$ is a continuous function at x_0 for all $x_0 \in (a, b)$ and for all $a, b \in \mathbb{R}$. To formalise this definition, we introduced a new variable x_0 and the restriction of $f : \mathbb{R} \rightarrow \mathbb{R}$ to the interval (a, b) . In particular, the function f passed to `continuous_x0` is restricted to (a, b) using the equivalent of a λ -function definition.

Observe that we need this non-standard definition of continuity on $\mathbb{R}$ to be able to use the notion of continuity defined in `continuous_x0`, which expects a function defined in some interval (a, b) . Obviously, Lean can handle continuity on $\mathbb{R}$ more cleanly by using the Mathlib definition `Continuous f` [2].

Next, we show that the function $g : \mathbb{R} \rightarrow \mathbb{R}$ given by $x \mapsto x$ has limit equal to x_0 at every point $x_0 \in \mathbb{R}$. We define this function via `def linear_fun : ℝ → ℝ := fun x => x`.

```

1  example : continuous_on_ℝ linear_fun := by
2  -- introduce assumptions that 'continuous_on_ℝ' expects.
3  intro a b x0 hx0
4  -- unfold continuity (and limit) at x0
5  dsimp [continuous_x0, limit_f_x_0]
6  intro ε hε
7  -- choose δ = ε and provide proof that δ > 0
8  refine ⟨ε, hε, ?_⟩
9  -- hx is the hypothesis: |x - x0| < ε and |x - x0| > 0
10 intro x hx
11 -- '[linear_fun]' replaces 'f' by its specific definition
12 simp [linear_fun] using hx.1

```

Listing 5: Example that $\lim_{x \rightarrow x_0} x = x_0$.

There are two tactics worth explaining in Listing 5. The first one is `dsimp`, which unfolds the definition of continuity on $\mathbb{R}$. This means writing out the definition of continuity at x_0 and, next, the definition of limit at x_0 . The second tactic is `refine ⟨ε, hε, ?_⟩`, which changes the goal by using $\delta := \varepsilon$ in the definition of limit at x_0 .

3.2 Definitions of boundedness

We now state the two definitions of boundedness on an interval in Listing 6 and prove their equivalence in Listing 7.

```

1  def bounded_on_interval_1 (f : Set.Ioo a b → ℝ) : Prop :=
2    ∃ (M m : ℝ), ∀ x : Set.Ioo a b, m ≤ f x ∧ f x ≤ M
3
4  def bounded_on_interval_2 (f : Set.Ioo a b → ℝ) : Prop :=
5    ∃ K : ℝ, 0 ≤ K ∧ ∀ x : Set.Ioo a b, |f x| ≤ K

```

Listing 6: Two equivalent definitions of boundedness.

```

1  lemma bounded_equiv (f : Set.Ioo a b → ℝ) :
2    bounded_on_interval_1 a b f ⇔
3    bounded_on_interval_2 a b f := by
4    -- separate the two implications
5    constructor
6    · -- (Def 1) → (Def 2)
7      intro h
8      -- unpack hypothesis and substitute
9      rcases h with ⟨M, m, hMm⟩
10     -- define bound for Def 2 with m, M
11     let K0 : ℝ := max (|m|) (|M|)
12     -- unpack what needs to be proved with K0
13     refine ⟨K0, ?_, ?_⟩
14     · -- prove K0 ≥ 0
15       have hm : 0 ≤ |m| := abs_nonneg m
16       have hmk : |m| ≤ K0 := le_max_left _ _ -- the underscores tell
17         Lean to infer the arguments
18       exact hm.trans hmk
19     · -- prove |f x| ≤ K0
20       intro x
21       -- prove -K0 ≤ f x
22       have hx_low : -K0 ≤ f x := by
23         -- use -K0 ≤ -|m| ≤ m ≤ f x
24         have h1 : -K0 ≤ -|m| := by
25           have : |m| ≤ K0 := le_max_left _ _
26           exact neg_le_neg this -- 'this' refers to the last 'have'
27           statement
28         have h2 : -|m| ≤ m := by linarith [neg_le_abs m]
29         have h3 : m ≤ f x := by simpa using (hMm x).1
30         exact h1.trans (h2.trans h3)
31       -- prove f x ≤ K0
32       have hx_up : f x ≤ K0 := by
33         -- f x ≤ M ≤ |M| ≤ K0
34         have hfxM : f x ≤ M := (hMm x).2
35         have hMabs : M ≤ |M| := le_abs_self M
36         have habsK : |M| ≤ K0 := le_max_right _ _
37         exact hfxM.trans (hMabs.trans habsK)
38       exact (abs_le.mpr ⟨hx_low, hx_up⟩) -- mpr uses the reverse
39         implication of 'abs_le'
40     -- the ⟨,⟩ brackets are used to construct a hypothesis of type A
41     ∧ B
42
43   · -- (Def 2) → (Def 1)
44     intro h
45     rcases h with ⟨K, Kpos, hK⟩
46     -- take m = -K, M = K, so, from |f x| ≤ K → -K ≤ f x ≤ K
47     refine ⟨K, -K, ?_⟩
48     intro x
49     -- abs_le.mp: |f x| ≤ K means -K ≤ f x ∧ f x ≤ K
50     simpa using (abs_le.mp (hK x)) -- mp uses the direct implication
51     of 'abs_le'

```

Listing 7: The two definitions of boundedness are equivalent.

There are a few tactics worth mentioning in Listing 7:

1. In line 13, the `refine` tactic is used in a different way. Here, this tactic creates two new goals associated with K_0 : prove that $K_0 \geq 0$ and $|f(x)| \leq K_0$. For this latter task, we break down the inequality in small pieces, which we prove in dedicated `have` statements.
2. In line 16, the underscores in `le_max_left _ _` tell Lean to infer the arguments for the lemma `le_max_left`, which states that $a \leq \max\{a, b\}$, for a, b in any linearly ordered type.
3. Throughout this snippet of code we extensively use the `trans` lemma, which gives a proof of the transitive property of non-strict inequalities.
4. In line 36 note how `.mpr` obtains the reverse implication in an “if and only if statement”, while `.mp` obtains the direct implication (line 46).
5. Finally, we use numerous lemmas regarding basic properties of the absolute value, which have been found in LeanSearch [1], using the `exact?` and `try?` tactics, or with GitHub copilot.

Next, we introduce the notion of boundedness in a subset $J \subseteq I$ as follows:

```
1 def bounded_on_J {a b : ℝ} (J : Set ℝ) (f : Set.Ioo a b → ℝ) : Prop :=
2   ∃ K : ℝ, 0 ≤ K ∧ ∀ x ∈ J, ∀ hx : x ∈ Set.Ioo a b, |f ⟨x, hx⟩| ≤ K
```

Listing 8: Boundedness of $f : J \subseteq I \rightarrow \mathbb{R}$.

3.3 Main Lemma

We can now prove the main Lemma of this project:

```
1 lemma bounded_if_limit_exists (f : Set.Ioo a b → ℝ) (x0 l : ℝ) (hx0ab :
2   x0 ∈ Set.Ioo a b) :
3   limit_f_x_0 l x0 f → ∃ δ > 0, bounded_on_J (Set.Ioo (x0 - δ) (x0 + δ))
4     f := by
5   intro hlim
6   -- use definition of limit with ε = 1
7   obtain ⟨δ, hδpos, hδ⟩ := hlim 1 (by norm_num)
8   -- define one possible bound
9   let K1 : ℝ := 1 + |l|
10  -- |f x| < K1 if |x - x0| < δ and |x - x0| > 0
11  have hδ_to_K1 :
12    ∀ x : Set.Ioo a b,
13    (|x - x0| < δ ∧ |x - x0| > 0) → |f x| < K1 :=
14  by
15    intro x hx
16    -- use triangle inequality to see |f x| ≤ |f x - l| + |l|
17    have triang : |f x| ≤ |f x - l| + |l| := by
18      -- norm_add_le does: |a + b| ≤ |a| + |b|
19      have := norm_add_le (f x - l) l
20      simp using this
21    -- |f x - l| < 1
22    have hfxl : |f x - l| < 1 := hδ x hx
23    -- transform to: |f x - l| + |l| < 1 + |l| = K1
24    have add_l : |f x - l| + |l| < K1 := by
25      -- add_lt_add_right does: if a < b then a + c < b + c
26      simp [K1] using add_lt_add_right hfxl |l|
27    -- conclude |f x| < K1
```

```

26     exact lt_of_le_of_lt triang add_1
27
28     -- define the final upper bound
29     let K2 : ℝ := max K1 |f ⟨x0, hx0ab⟩|
30     -- simplify hypothesis and split goals
31     refine ⟨δ, hδpos, ?_⟩
32     refine ⟨K2, ?_, ?_⟩
33     · -- prove 0 ≤ K2
34       -- prove 1 ≤ K1 = 1 + |ℓ|, since |ℓ| ≥ 0
35       have h1leqK1 : 1 ≤ K1 := by
36         have habs : 0 ≤ |ℓ| := abs_nonneg 1
37         -- 1 ≤ 1 + |ℓ|
38         linarith [habs]
39       -- prove K1 ≤ K2 = max K1 |f x0|
40       have hK1leqK2 : K1 ≤ K2 := by
41         simpa [K2] using le_max_left K1 |f ⟨x0, hx0ab⟩|
42       exact (le_trans (by norm_num) (h1leqK1.trans hK1leqK2))
43     · -- prove |f x| ≤ K2 for x ∈ (x0 - δ, x0 + δ)
44     intro x hxI hxDom
45     by_cases hxeq : x = x0 -- observe x satisfies both hxI and hxDom
46     · -- bound the case x = x0
47       have hx0_le_K2 : |f ⟨x0, hx0ab⟩| ≤ K2 :=
48         le_max_right _ _
49       have : |f ⟨x, hxDom⟩| ≤ K2 := by
50         simpa [hxeq] using hx0_le_K2
51       exact this
52     · -- bound the case x ≠ x0, i.e., |x - x0| < δ and 0 < |x - x0|
53     have hx_abs_lt : |x - x0| < δ := by
54       have hL : -δ < x - x0 := by linarith [hxI.1]
55       have hR : x - x0 < δ := by linarith [hxI.2]
56       exact abs_lt.mpr ⟨hL, hR⟩
57     have hpos_abs : |x - x0| > 0 := abs_pos.mpr (sub_ne_zero.mpr hxeq)
58     -- bound |f x| < K1 for x ∈ 0 < |x - x0| < δ
59     have fLK1 : |f ⟨x, hxDom⟩| < K1 :=
60       -- use |x - x0| < δ and 0 < |x - x0|
61       hδ_to_K1 ⟨x, hxDom⟩ ⟨by simpa using hx_abs_lt, hpos_abs⟩
62     -- K1 ≤ K2
63     have K1leK2 : K1 ≤ K2 := by
64       simpa [K2] using (le_max_left _ _)
65     -- conclude |f x| ≤ K2
66     exact (le_of_lt fLK1).trans K1leK2

```

Listing 9: A function with limit at x_0 is bounded on some interval around x_0 .

The tactics and lemmas we use in [Listing 9](#) are very similar to the ones we described in [Listing 7](#). In the following lines I will outline the general structure of the main Lemma's proof.

1. We use the definition of limit with $\varepsilon := 1$ and use the second definition of boundedness. Using this definition, we first need to prove that $|f(x)| < K_1 := 1 + |\ell|$ for all $x \in (x_0 - \delta, x_0 + \delta)$ except for $x = x_0$.
2. We prove this by first seeing that $|f(x)| \leq |f(x) - \ell| + |\ell|$ (through the triangle inequality), next using that $|f(x) - \ell| < 1$, and finally building the sequence $|f(x)| \leq |f(x) - \ell| + |\ell| < 1 + |\ell| = K_1$.
3. Since we also need to consider the point x_0 , we define $K_2 := \max\{K_1, |f(x_0)|\}$, and prove that K_2 is the non-negative constant that satisfies the definition of boundedness in $(x_0 - \delta, x_0 + \delta)$.
4. For this goal, we need to prove two goals (through the `refine` tactic), the first of which being to prove that $K_2 \geq 0$. We do this by proving that $1 \leq K_1 = 1 + |\ell|$ and that $K_1 \leq K_2 = \max\{K_1, |f(x_0)|\}$.

5. The second and last goal is to prove that $|f(x)| \leq K_2$ for all $x \in (x_0 - \delta, x_0 + \delta)$. The `by_cases` tactic allows us to separate the cases $x = x_0$ and $x \neq x_0$. The first case is straightforward, and the second requires showing that $|f(x)| < K_1 \leq K_2$.

Finally, we run the `#lint` command to check for formatting, and it shows that all the linting tests have passed, and no errors occur.

4 Challenges faced

I found it particularly tricky to understand the correct use of the symbols “{ }” and “()” when used as hypotheses. This was particularly noticeable in the definition of the limit of a function, where the following definition failed when called inside the main Lemma.

```

1   def limit_f_x_0 (a b : ℝ) (l x_0 : ℝ) (f : Set.Ioo a b → ℝ) : Prop :=
2     ∀ ε > 0, ∃ δ > 0, ∀ x : Set.Ioo a b, |x - x_0| < δ ∧ |x - x_0| > 0 → |f
      x - l| < ε

```

Listing 10: (Incorrect) definition of limit at x_0 .

However, changing `(a b : ℝ)` to curly brackets `{a b : ℝ}` makes all the difference, since this produces no “application type mismatch”. Looking at the Lean documentation, this behaviour occurs because the arguments inside curly brackets are not to be specified when calling the definition somewhere else in the code.

Regarding error messages, I also found it particularly tricky to interpret them, as they appear (to my unexperienced eye) not strongly related to the true source of the error. In fact, Dr. Kevin Buzzard told us once that the Lean community knows about this issue and is actively seeking to make error messages more understandable.

Formalising the results in [Section 2](#) has also drawn my attention to details that I had overlooked before. For instance, a subtle point I realised is the interval where $f(x)$ is bounded on. When I was writing the informal proof I thought it would be enough to define what “bounded on (a, b) ” is (see [Listing 6](#)). However, when I was writing the proof of the main Lemma in Lean, I realised I needed to prove that f is bounded on some neighbourhood $(x_0 - \delta, x_0 + \delta)$ of x_0 , which is not necessarily (a, b) . Thus, I had to introduce a general definition of boundedness on an interval I and then introduce the idea of f being bounded on some interval $J \subseteq I$ (see [Listing 8](#)). Obviously, in the Lean code we set $I := (a, b)$.

I also found it hard to find the right result to conclude the proof of the main Lemma in [Section 3](#). I had the hypothesis `hltK1 : |f⟨x, hxDom⟩| < K1` and `K1_le_K2 : K1 ≤ K2` and I wanted to conclude that `f |f⟨x, hxDom⟩| ≤ K2`. Since the `exact?` tactic was not giving any useful results, I looked for the right lemma in the “LeanSearch” tool [\[1\]](#), as one TA suggested in the labs. The correct tactic to use here is `exact (le_of_lt hltK1).trans K1_le_K2`, where `le_of_lt` gives the hypothesis `|f⟨x, hxDom⟩| ≤ K1` and `.trans` applies transitivity on non-strict inequalities.

On a positive note, I found surprisingly easy writing the mathematical definitions. I also found useful the built-in search engine for lemmas in Lean (i.e., the `exact?` and `try?` instructions). Moreover, I found quite surprising the fact that the `#lint` command picks up on missing docstrings for definitions or theorems. I added these to resolve the errors `#lint` was showing at first.

4.1 Reflecting on the chosen lemma

The result on boundedness of functions around a point was not the one I initially intended to prove. At first I tried to prove the following:

Lemma: Finite Jensen's Inequality

Let $n \geq 2$ and $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function. If $\alpha_1, \dots, \alpha_n \in [0, 1]$ satisfy $\alpha_1 + \dots + \alpha_n = 1$, then the inequality

$$\varphi(\alpha_1 x_1 + \dots + \alpha_n x_n) \leq \alpha_1 \varphi(x_1) + \dots + \alpha_n \varphi(x_n)$$

holds for all $x_1, \dots, x_n \in \mathbb{R}$.

This Lemma can be easily proved by induction on the number of points n . I broke down the proof in small lemmas that were easier by themselves. Once I had the small pieces formalised, however, I was unable to put together the induction step. I read about the induction' tactic but was not able to use it satisfactorily to produce the final step.

5 Conclusions

In all, this has been an instructive introduction to Lean, where I have learnt to define concepts and to apply them to simple examples and easy lemmas. I have also proved the equivalence of two definitions of boundedness around a point, and further formally proved that if f has a limit in x_0 , then f is bounded on some neighbourhood of x_0 .

For the upcoming projects I aim to prove some lemma or theorem that requires induction, since I did not succeed in my first attempt to prove Jensen's inequality. I am also interested in exploring other areas of mathematics such as Galois Theory or Complex Analysis, with an expectation to learn more tactics and discover the Mathlib library in greater depth.

References

- [1] BICMR@PKU AI4M Team. LeanSearch, Find theorems in Mathlib4 using natural language query. URL: <https://leansearch.net/>.
- [2] Lean Community. Mathlib 4 Documentation. URL: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Topology/Defs/Basic.html#Continuous.
- [3] Francesc Mañosas. Lecture notes on real-valued functions. 1st year mathematics undergraduate course, Autonomous University of Barcelona, 2019.